

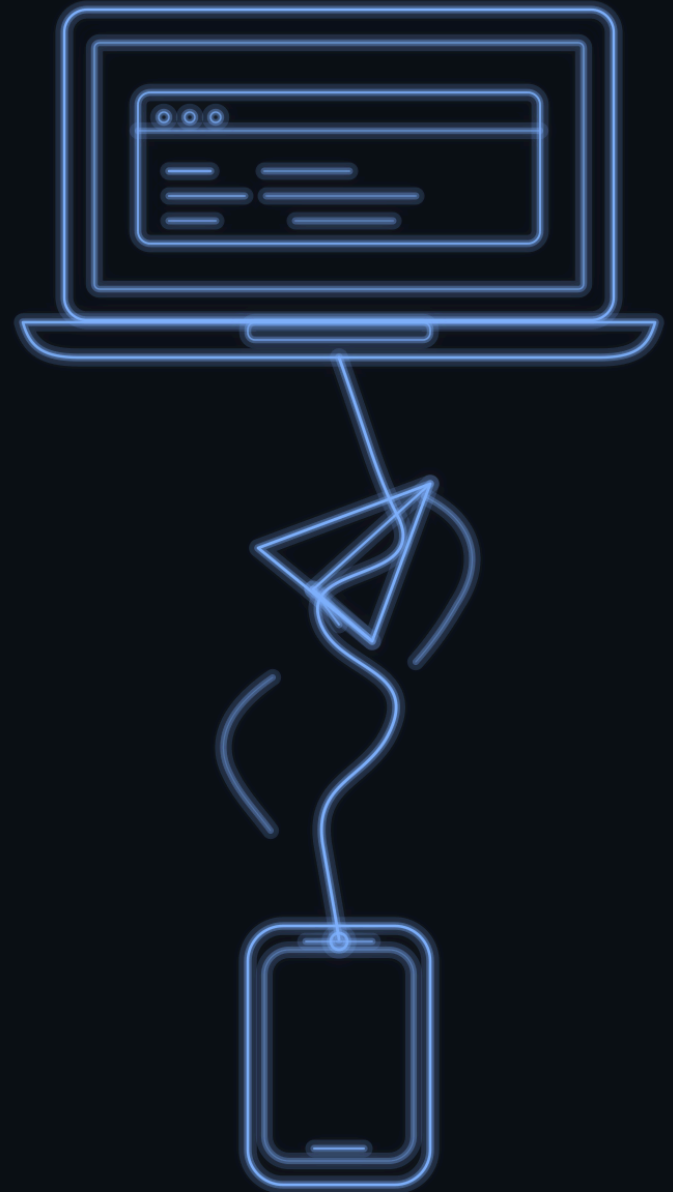
The Most Fragile Line of Code

Controlling an AI Agent From Your Phone

Robert E. Lee

github.com/robertelee78/claude-telegram-mirror

linkedin.com/in/robertgpt



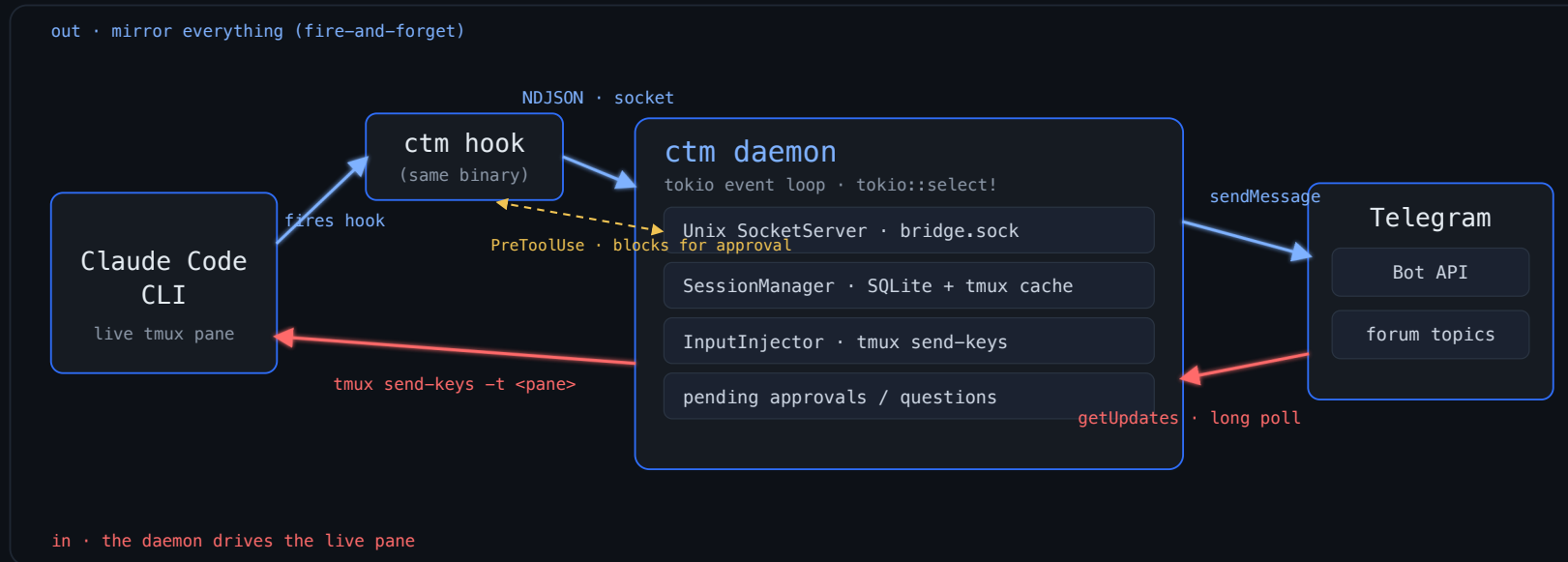
February. Fourth kid. "Paternity leave."

- An AI agent running on my machine, **mid-task** — the kind you don't leave unsupervised.
- The other room: my wife needs a hand with the baby. **Now.**
- Leave the terminal → an agent runs wild on my code. Stay → I leave her holding the baby.

*I needed to be able to see everything it was doing — every command, every edit, not just the questions — and catch it the instant it wandered off, to **interrupt midstream and redirect**. From my phone. As if I were still sitting there.*

In 10 minutes I'll do exactly that, live. First: the line of code that made it possible was, for months, the most fragile in the project.

ctm — a Rust bridge: Claude Code $\rightleftharpoons$ Telegram



The whole point isn't two *copies* of the conversation. It's *one* conversation I can touch from either side.

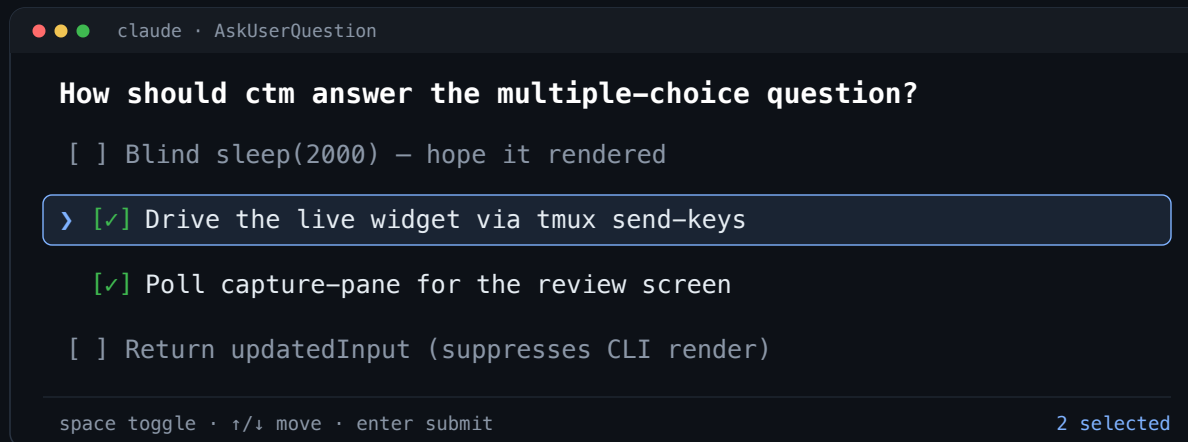
Desk → walk away → answer on my phone → sit back down and keep typing. Same session, mid-thought. **I never switch contexts. The interface follows me.**

~20,000 lines of Rust · 15 ADRs · ~595 tests · ~6 months.

Out is easy. *In* is brutal.

- **Out:** the agent fires events — a *hook* (a script Claude runs at lifecycle moments). I forward them. Fire and forget.
- **In:** no input API. Claude Code reads a **terminal** — the only door. So *anything* I'd type at the desk — a prompt, a command, `stop` — I inject from my phone (`tmux send-keys`).
- Free text is the easy case. A yes/no approval the hook hands straight back. The **brutal** one: a **multiple-choice question** — an interactive widget, arrow keys and all.

You can't pipe text at that. You have to drive it.



```
claude · AskUserQuestion

How should ctm answer the multiple-choice question?

[ ] Blind sleep(2000) - hope it rendered
> [✓] Drive the live widget via tmux send-keys
[✓] Poll capture-pane for the review screen
[ ] Return updatedInput (suppresses CLI render)

space toggle · ↑/↓ move · enter submit                2 selected
```

v1: I pretended to be a human

```
let key_delay = Duration::from_millis(300);

for &idx in selected_indices {           // toggle each chosen option
    inj.send_key(&format!("{}", idx + 1));
    sleep(key_delay).await;
}
let downs_needed = total_options + 2;    // walk down to "Submit"
for _ in 0..downs_needed {
    inj.send_key("Down");
    sleep(key_delay).await;
}
inj.send_key("Enter");
```

I built a robot... to hunt-and-peck... for another robot. **~9 keys × 300ms ≈ 2.7s** — then I had to wait for the review screen before pressing Enter. How long does that take? **I had no idea. So I guessed.**

sleep(2000ms)

// 2s is enough

~4.7s per answer — riding on a guess. When 2s wasn't enough, I pressed Enter into a screen that hadn't rendered yet, and the answer just... evaporated.

Then: stop typing. Speak the protocol.

- One kind of hook can **block** and hand back structured data.
- So: don't impersonate a user. Intercept the question, collect the answer in Telegram, hand it **straight back as JSON**.
- No terminal. No keystrokes. No `Down, Down, Down`. No guessing how long a screen takes to paint.

That ~4.7-second dance collapsed into building a tiny answer object — **microseconds to assemble**. I ripped out the whole keystroke path.

*It felt fantastic. It felt a little **too** clean. Hold that thought.*

The fix forced the exact thing it was meant to kill

Handing the answer back as data tells Claude: *"already answered — don't ask."*

So it **stops drawing the question in the terminal at all**.

- The question now lives **only** in Telegram.
- Sitting right at my keyboard? I *can't answer where I am*. I have to stop and pick up my phone.
- I built this so I'd **never** switch interfaces... and the "elegant" version **mandated** one. Every time.

*It didn't just drop a surface. It broke the **seamlessness** — the one continuous conversation — that's the entire reason the thing exists.*

So I put the keystrokes back — and fixed the line that was *actually* broken

- Injection was never a hack here. It's the **one mechanism that keeps the question on both screens**. It was load-bearing the whole time.
- The broken line wasn't the *layer*. It was the **guess** — *every* keystroke fired blind, hoping the screen had caught up.

```
- send_key(..); sleep(300ms)      // navigate blind, hope the TUI kept up
- sleep(2000ms); press(Enter)    // hope the review screen exists yet
+ loop {                        // read → verify → act, every step
+   let view = parse_widget(capture_pane()); // what's ACTUALLY on screen?
+   if view.cursor_on(target) { break }      // verified before any key
+   send_key("Down")
+ } // Enter only on a verified confirm screen — else fail closed
```

It stopped *guessing* and started **checking — every step**. And when a keystroke can't be verified, it **fails closed** instead of corrupting a half-driven widget.

Live: my phone quizzes me

Laptop terminal + iPhone, side by side.

From the phone: *"ask me a hard multiple-choice trivia question."*

The question renders in **both** — terminal *and* phone. Tap the answer on the phone → the **terminal** advances.

[ASSET: ≤90s side-by-side recording → assets/demo.mp4 — also the stage fallback]

Three things I took from this

1. **A fragile line isn't always at the wrong layer — sometimes it's just *unverified*.** Fix the verification before you fix the architecture.
2. **An elegant rewrite can optimize away the product.** Know what a line is load-bearing *for* before you delete it.
3. **"Elegant" and "correct" are different measurements.** The clean version won the benchmark and lost the point. Measure the regression, not just the speedup.

One footnote, then I'll stop

*While I was deep in this, Anthropic shipped **Channels** — an official Claude Code → **Telegram** bridge. Good validation, and genuinely good: its live mode even relays **yes/no** approvals to phone and terminal at once — first answer wins. Two things it still doesn't do: it never mirrors the **multiple-choice** question (that stays terminal-only), and it tells you to "run separate sessions" — one bot, one session. `ctm` puts the multiple-choice question on **both** screens and multiplexes every session on the box into its own Telegram topic. The headless shortcut for the hard part? It just **turns the questions off**. I didn't want to give that up.*

The fix was never faster keystrokes.

Sometimes the most fragile line isn't in the wrong place — it just never checked whether the thing it was waiting for had actually happened.

`github.com/robertlee78/claude-telegram-mirror` · Robert E. Lee · `linkedin.com/in/robertgpt`

Appendix — the receipts

Project	<code>claude-telegram-mirror</code> (<code>ctm</code>), ~20,211 lines of Rust (src), 15 ADRs, ~595 tests
v1 (fragile)	300ms/key; ~9-key multi-select $\approx$ 2.7s + a blind <code>sleep(2000)</code> "2s is enough" $\Rightarrow$ ~4.7s/answer, racing the review screen
v2 (false summit, ADR-014 PR-E)	answer via <code>updatedInput</code> ; μ s to build — <i>but</i> <code>updatedInput</code> suppresses the CLI render (+ the <code>tool_start</code> Telegram signal) $\rightarrow$ broke both-surfaces
v3 (real fix, ADR-015)	injection restored; <code>tmux capture-pane</code> readiness polling (<code>READY_POLL_INTERVAL_MS=200</code> , 25-poll / 5s cap), Enter only on the verified review screen; no blind race
Channels (research preview, v2.1.80+)	Official Claude Code $\rightarrow$ Telegram bridge (an MCP plugin spawned over stdio). Live mode relays <code>Bash</code> / <code>Write</code> / <code>Edit</code> <i>yes/no approvals to terminal and phone — first-answer-wins</i> (v2.1.81+) — but does not mirror the AskUserQuestion multiple-choice widget (terminal-only), and the docs say <i>"to process independent event streams concurrently, run separate sessions"</i> (one bot $\approx$ one session). Headless <code>-p</code> : <i>"tools that need terminal input, such as multiple-choice questions and plan mode approval, are disabled so the session never stalls."</i>

ADR-014 $\rightarrow$ ADR-015 (supersedes PR-E) in `docs/adr/`. Claude Code render-suppression contract: anthropics/claude-code #29547. Channels: `code.claude.com/docs/en/channels-reference`.